**FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES**
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
*AL-2002 Artificial Intelligence (Laboratory) · Fall 2026*

# PROJECT PROPOSAL

*Semester Lab Project · Weightage 1 mark*

*Instructions — delete this box before submitting. Complete every section below in three to four pages. Replace all italicised blue guidance text with your own writing, and delete any row or prompt you do not need. Submit this document in BOTH PDF and Markdown formats, placed in the docs/ folder of your GitHub repository, and have the group leader submit the repository link to Google Classroom.*

## Group Information

| | Name | Roll Number | Section | Role in the project |
|---|---|---|---|---|
| **Leader** | Haji Muzammil | 24k-2057 | BCY-5A | Leader / Developer |
| Member 2 | Zakir Hussain | 24k-2045 | BCY-5A | Developer |
| Member 3 | Muhammad Osama | 24k-2030 | BCY-5A | Developer / Analyst |

| | |
|---|---|
| **Project Title** | Delivery Route Optimizer on a Real City Map |
| **Project Idea** | Idea No. 14 from the Project Ideas list ☑ Own idea (needs approval) ☐ |
| **Category** | Part A — Game ☐ Part B — Applied / Data ☑ |
| **GitHub Repository URL** | https://github.com/Haji-Muzammil/Delivery-Route-Optimizer |
| **Date of Submission** | October 21, 2026 |

## 1. Project Title and Summary

*State the title of your project, then summarise it in four to six sentences: what the software does, who would use it or play it, and what AI makes possible in it that a plain program could not do. Write it so that a reader who knows nothing about your topic understands the idea by the end of the paragraph. Avoid listing libraries here.*

Delivery Route Optimizer on a Real City Map

This project is an intelligent route optimization application that computes the most efficient path for a delivery driver navigating through 10 to 20 delivery addresses in a real city. Instead of relying on abstract toy graphs, the software fetches real road network data for cities like Lahore, Karachi, or Islamabad using OpenStreetMap data via OSMnx. It will be utilized by logistics operators or delivery riders seeking to minimize total travel distance and fuel consumption. To accomplish this, the system applies the A* search algorithm to compute exact shortest driving paths between individual waypoints, and subsequently deploys local search techniques—including Simulated Annealing and Genetic Algorithms—to solve the overarching Traveling Salesperson Problem (TSP). Ultimately, an interactive Folium map visualizes the computed optimal driving route for the driver.

## 2. Problem Statement and Motivation

*Describe the problem your project addresses in 120–180 words. Why is it hard to solve by hand or by brute force — how large is the search space, how uncertain is the information, how many constraints interact? Who faces this problem, and how do they cope today? Finish with one sentence on why solving it is worthwhile. Be specific — "puzzles are hard" says nothing; "a 9×9 Sudoku has around 10^21 possible fillings of its empty cells, so blind enumeration is hopeless" says a great deal.*

The project addresses the routing challenge inherent in 'last-mile' delivery logistics, formulated as a Traveling Salesperson Problem (TSP) on a real-world road network graph. It is exceedingly difficult to solve by brute force because the search space of visiting sequences grows factorially; for just 15 delivery stops, there are over 87 billion (15!) possible permutations. Furthermore, evaluating the true driving distance between any two stops requires pathfinding across thousands of real geographical intersections (nodes) and road segments (edges). Delivery riders, courier services, and postal workers face this daily, currently coping through static route familiarity or manually plotting stops on consumer GPS apps that do not automatically optimize a multi-stop sequence. Solving this is highly worthwhile because optimized routing significantly reduces fuel consumption, carbon emissions, and operational costs, proving how AI search delivers practical economic value.

## 3. Problem Formulation

*Describe your problem in terms of its agent and environment. Every later design decision follows from this section, so be precise.*

### 3.1 PEAS Description

| Component | Description for your project |
|---|---|
| **Performance measure** | Total route distance in kilometers and algorithm execution time in seconds. |
| **Environment** | Real-world road network graph extracted via OpenStreetMap (OSMnx). |
| **Actuators** | Generated optimal route sequence and interactive Folium HTML map visualization. |
| **Sensors** | Array of geographical coordinates (latitude/longitude) representing delivery stops. |

### 3.2 Environment and Agent Type

| Property | Classification — tick one |
|---|---|
| Observability | Fully observable ☑ Partially observable ☐ |
| Determinism | Deterministic ☑ Stochastic ☐ |
| Change over time | Static ☑ Dynamic ☐ |
| State and actions | Discrete ☑ Continuous ☐ |
| Agents | Single-agent ☑ Multi-agent / adversarial ☐ |
| Agent type | Simple reflex ☐ Model-based ☐ Goal-based ☑ Utility-based ☐ Learning ☐ |

### 3.3 Problem Representation

*For a search, game or CSP project, fill every row. For a data-driven project, use the state row for the input features and the goal row for the target you will predict.*

| Element | Your representation |
|---|---|
| **State representation** | An ordered list/array of delivery stop node IDs representing the current route permutation. |
| **Actions / moves** | Swapping two delivery stops in the sequence, or mutating a sub-segment of the route. |
| **Initial state** | A randomly generated valid permutation sequence of all delivery stops. |
| **Goal or terminal test** | Reaching the maximum iteration/generation limit or temperature cooling to zero. |
| **Cost, utility, fitness or evaluation** | Total cumulative driving distance of the route computed via pre-calculated A* paths (lower is better). |

# 4. Proposed AI Approach

*Name each AI technique you will implement and what it does in your project. The core technique must be compared against an alternative — a second technique, a second configuration, or a simple baseline. State whether each is your own code or a library.*

| # | Technique | Role in the project | Own code or library |
|---|---|---|---|
| 1 | A* Search | Calculates exact shortest driving distance between pairs of stops on the road graph. | Own code |
| 2 | Simulated Annealing | Explores the TSP route permutation space, accepting worse moves initially to escape local minima. | Own code |
| 3 | Genetic Algorithm | Evolves a population of route permutations using crossover and mutation operators. | Own code |
| B | Baseline: Hill Climbing | Greedy local search that only accepts route swaps that strictly decrease total distance. | Own code |

*In 100–150 words, explain why these techniques suit the problem formulated in Section 3, and describe your heuristic, evaluation function, fitness function or probabilistic model in plain words.*

These techniques suit the problem because it involves two distinct search layers. The first layer is point-to-point pathfinding on a road graph; A* is optimal here using Haversine straight-line distance as an admissible heuristic. The second layer is optimizing the visiting order of 15–20 stops (TSP). Since exact TSP solvers scale poorly, local search algorithms (Simulated Annealing, Genetic Algorithm, and Hill Climbing) provide excellent near-optimal solutions. To ensure high performance, A* is used to pre-compute an N×N distance matrix once, allowing local search agents to evaluate route fitness in O(1) time.

## 4.1 Planned Class Design

*List the classes you intend to write — at least three — and for each give its main attributes and methods and its relationship to the others (for example an abstract Agent with RandomAgent and AStarAgent subclasses). A simple indented list is acceptable at proposal stage; the full diagram belongs in the report.*

• MapEnvironment: Loads and holds the OSMnx graph. Key methods: load_graph(), get_random_nodes(), save_cache().
• PathFinder: Implements point-to-point pathfinding. Key methods: astar_search(), haversine_heuristic(), build_distance_matrix().
• OptimizerAgent (Abstract Base Class): Defines the TSP interface. Attributes: distance_matrix, best_route, best_cost. Key methods: optimize().
• HillClimbingAgent (inherits OptimizerAgent): Implements greedy hill climbing route search by neighbor swapping.
• AnnealingAgent (inherits OptimizerAgent): Implements simulated annealing with exponential cooling schedule.
• GeneticAgent (inherits OptimizerAgent): Implements population-based GA with order crossover (OX) and swap mutation.

## 5. Scope

**In scope — the features you commit to delivering.** List the features that must work for the project to be considered complete.

| # | Feature | Why it is essential |
|---|---|---|
| 1 | Real Road Network Integration via OSMnx | Transforms abstract graph search into a practical real-world applied scenario. |
| 2 | Custom A* Search Implementation | Computes realistic driving distances instead of straight-line Euclidean estimates. |
| 3 | Hill Climbing, SA, and GA Solvers | Fulfills the core AI requirement of comparing multiple search techniques against a baseline. |
| 4 | Interactive Folium Map Visualization | Renders the optimized path clearly on a real geographic map layout. |
| 5 | Offline Data Caching Logic | Ensures the application runs smoothly without internet dependency during evaluation. |

**Out of scope — deliberately excluded.** Naming what you will not build is as important as naming what you will; it shows that your scope is realistic for a group of two.

Real-time dynamic traffic updates, turn-by-turn navigation voice guidance, multi-vehicle fleet routing (VRP), and dynamic mid-route stop insertions are deliberately excluded to keep the project focused on core search algorithms.

## 6. Data or Environment Source

*Identify exactly where your data will come from, and confirm that you can actually obtain it. If your project is a game, puzzle or simulation with no external data, tick "Generated / simulated" and describe how problem instances — boards, maps, puzzles, sensor readings — are generated and seeded.*

| | |
|---|---|
| **Source name and URL** | OpenStreetMap road graph data fetched via Python OSMnx library (https://openstreetmap.org) |
| **Type** | Public dataset ☑ Web API ☐ Self-collected ☐ Generated / simulated ☐ |
| **Access requirements** | No key needed ☑ Free key required ☐ (Paid sources are not permitted) |
| **Approximate size** | 10–50 MB per downloaded city bounding box network (.graphml format) |
| **Licence / terms of use** | Open Data Commons Open Database License (ODbL) |
| **Offline fallback** | Downloaded city road graphs are saved locally as .graphml files in data/raw/. The application loads these cached files when network access is unavailable. |

***Required:*** your program must still run when the network is unavailable, by reading a locally cached copy of any data it downloads. Describe your caching plan in the last row above.

## 7. Evaluation Plan

*Explain how you will prove that your AI works. Typical metrics are win-rate, average score, nodes expanded, run time, solution cost, accuracy and RMSE. The same instances, the same metric and a fixed seed must be used for every approach you compare.*

| | |
|---|---|
| **What will be compared** | Hill Climbing (Baseline) vs. Simulated Annealing vs. Genetic Algorithm |
| **Metric(s)** | Total route distance (km), execution time (seconds), and convergence rate |
| **Baseline** | Hill Climbing (greedy local search) |
| **Test instances / test set** | 3 benchmark city maps (Karachi, Lahore, Islamabad) evaluated across 10, 15, and 20 delivery stops |
| **Number of runs and seed** | 10 runs per test instance using fixed random seed = 42 |
| **Planned charts (at least three)** | 1. Route Distance Comparison Bar Chart<br>2. Iteration vs. Fitness Convergence Line Plot<br>3. Interactive Folium Route Map Overlay |

## 8. Division of Work

*State clearly which member is responsible for which part. This table will be compared against your GitHub commit history and against the contribution table in the final report, so make it honest and keep it updated.*

| Member | Modules / components owned | Approximate share |
|---|---|---|
| Haji Muzammil | Map environment integration (OSMnx), A* pathfinding algorithm, distance matrix pre-computation, offline caching system (src/environment.py, src/search.py). | 34% |
| Zakir Hussain | TSP Optimizer Agents (Hill Climbing, Simulated Annealing, Genetic Algorithm) (src/agents.py). | 33% |
| Muhammad Osama | Experiment execution runner, benchmark testing, Folium map visualization, result charts (src/experiments.py, src/visualise.py). | 33% |

## 9. Milestones and Timeline

*Break the work into weekly or fortnightly milestones from now until the submission deadline in Week 15, one week before the Lab Final Exam. Each milestone should name something demonstrable, not an activity — "A* solves all 20 test mazes and prints nodes expanded" rather than "work on search".*

| # | Milestone — what will be demonstrably working | Target date | Owner |
|---|---|---|---|
| 1 | OSMnx map graph loading with offline caching and basic Folium map rendering working. | Week 11 | Haji Muzammil |
| 2 | Custom A* search algorithm computing node-to-node shortest paths and filling distance matrix. | Week 12 | Haji Muzammil |
| 3 | Hill Climbing baseline and Simulated Annealing agents optimizing 15-stop routes. | Week 13 | Zakir Hussain |
| 4 | Genetic Algorithm agent running; benchmark experiment runner generating comparison charts. | Week 14 | Muhammad Osama |
| 5 | Experiments complete, final integration, documentation and repository cleanup. | Week 15 | All Members |

## 10. Expected Outcome and Risks

*In 80–120 words, describe what a working submission will look like — what a user will see when they run it, and what results you expect to report (for example, "A* expands roughly a tenth of the nodes BFS expands"). Then name the two risks most likely to derail the project — for example the search being too slow at the depth you need, or the dataset proving too small to train on — and state what you will do if each occurs.*

A working submission will prompt the user to select a city and number of delivery stops. The system will compute the optimal visiting order, display terminal logs of distance reduction per algorithm, and open an interactive Folium map tracing the complete path. We expect Simulated Annealing and GA to achieve 15–25% shorter route distances than Hill Climbing.

Risk 1: Fetching large map graphs from OpenStreetMap live may fail or time out during evaluation.
Mitigation: Pre-download and commit cached .graphml files in data/raw/ so the program runs entirely offline.

Risk 2: Re-running A* for every fitness check in GA/SA would cause severe computational slowdown.
Mitigation: Pre-compute an N×N distance matrix using A* once before optimization, allowing O(1) fitness lookups.

| **For office use — Lab Instructor** | |
|---|---|
| Proposal status | Approved ☐ Approved with changes ☐ Revision required ☐ |
| Comments | |
| Signature and date | |

*End of Project Proposal*